

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ИНДИВИДУАЛЬНОЕ ДОМАШНЕЕ ЗАДАНИЕ
по дисциплине «Введение в нереляционные СУБД»
Тема: Сервис отслеживания криптовалют

Студент гр. 3342

Студенты гр. 3343

Преподаватель

Песчатский С.Д.

Бондаренко Ф.А.

Пименов П.В.

Заславский М.М.

Санкт-Петербург

2026

ЗАДАНИЕ

Студент группы 3342

Песчатский С.Д.

Студенты гр. 3343

Бондаренко Ф.А.

Пименов П.В.

Тема работы: Сервис отслеживания криптовалют

Исходные данные:

Задача — создать сервис для отслеживания портфеля криптовалют с актуальными котировками, историческими данными, интерактивными графиками и анализом динамики цен. Данные хранить в MongoDB.

Полный текст задания: [ИДЗ - состав, порядок работы \[МОЭВМ Вики se.moevm.info\]](#)

Содержание пояснительной записки:

- Содержание
- Введение
- Сценарии использования
- Модель данных
- Разработанное приложение
- Заключение
- Приложения
- Литература

Предполагаемый объем пояснительной записки:

Не менее 20 страниц.

Дата выдачи задания: 13.02.2026

Дата сдачи реферата: 21.05.2026

Дата защиты реферата: 21.05.2026

Студент гр. 3342

Студенты гр. 3343

Преподаватель

Песчатский С.Д.

Бондаренко Ф.А.

Пименов П.В.

Заславский М.М.

АННОТАЦИЯ

В рамках данного курса предполагалось разработать какое-либо приложение в команде на одну из поставленных тем. Была выбрана тема создания веб-приложения для отслеживания портфеля криптовалют с актуальными котировками, историческими данными и интерактивным анализом. Приложение позволяет пользователям фильтровать и сортировать монеты, сравнивать динамику цен с разной агрегацией данных (часы, дни, недели), сохранять персональные настройки и выполнять массовый импорт/экспорт (администраторам). Требуется реализовать хранение данных в нереляционной СУБД MongoDB. Найти исходный код и всю дополнительную информацию можно по ссылке: <https://github.com/moevm/nsql1h26-crypto>

SUMMARY

Within the framework of this course, a team was supposed to develop an application on one of the proposed topics. The topic of creating a web application for tracking a cryptocurrency portfolio with up-to-date quotes, historical data, and interactive analysis was chosen. The application allows users to filter and sort coins, compare price dynamics with different data aggregation periods (hours, days, weeks), save personal settings, and perform bulk import/export (for administrators). The task requires implementing data storage in the MongoDB. The source code and all additional information can be found at: <https://github.com/moevm/nsql1h26-crypto>

СОДЕРЖАНИЕ

1.	Введение	6
1.1.	Актуальность решаемой проблемы	6
1.2.	Постановка задачи	6
1.3.	Предлагаемое решение	8
1.4.	Качественные требования к решению	8
2.	Сценарии использования	10
2.1.	Макет UI	10
2.2.	Сценарии использования	10
3.	Модель данных	30
3.1.	Нереляционная модель	30
3.2.	Реляционная модель	48
3.3.	Сравнение моделей	63
3.4.	Вывод	65
4.	Разработанное приложение	66
4.1.	Краткое описание	66
4.2.	Использованные технологии	68
4.3.	Ссылка на приложение	68
4.4.	Снимки экрана приложения	68
5.	Заключение	71
5.1.	Достигнутые результаты	71
5.2.	Недостатки и пути для улучшения полученного решения	72
5.3.	Будущее развитие решения	73
6.	Приложения	74
6.1.	Документация по сборке и развертыванию приложения	74
7.	Список использованных источников	75

ВВЕДЕНИЕ

1. Введение

1.1. Актуальность решаемой проблемы

Рынок криптовалют характеризуется высокой волатильностью, большим количеством активов и непрерывным потоком обновляющихся рыночных данных. В таких условиях пользователям трудно оперативно отслеживать состояние собственного портфеля, сопоставлять монеты по ключевым метрикам и принимать обоснованные решения на основе исторической динамики. Существующие решения нередко либо перегружены избыточной функциональностью, либо не обеспечивают достаточной гибкости в фильтрации, сравнении и персонализации аналитики. Дополнительно усложняет задачу необходимость надежного хранения пользовательских настроек, разграничения прав доступа и безопасного обмена данными между средами. В связи с этим актуальной является разработка веб-приложения, объединяющего хранение, представление и анализ данных по криптовалютам: с поддержкой актуальных котировок, исторических срезов, интерактивной визуализации, пользовательских пресетов аналитики, а также административных функций импорта/экспорта. Такой сервис повышает доступность аналитики, ускоряет работу с данными и обеспечивает более контролируемую эксплуатацию системы.

1.2. Постановка задачи

Цель работы

Предоставить пользователям и администраторам веб-сервис для мониторинга и анализа криптовалют, обеспечивающий отслеживание портфеля, поиск, фильтрацию, сортировку и сравнение монет, построение пользовательской статистики с различными периодами агрегации, а также сохранение персональных настроек и управление данными через механизмы

импорта/экспорта. Реализация должна включать REST API на Java/Spring Boot, хранение данных в MongoDB (включая пользовательские сущности и исторические снимки котировок), адаптивный клиентский интерфейс на Next.js и возможность синхронизации с внешним API CoinMarketCap при сохранении сценария работы на локальных данных.

Задачи

- Разработать систему аутентификации и авторизации пользователей: Реализовать регистрацию и вход пользователей, а также разграничение прав доступа по ролям (пользователь/администратор) для защиты пользовательских и административных сценариев.
- Реализовать модуль отслеживания портфеля и управления списками монет: Обеспечить работу с watchlist и избранным, включая добавление/удаление монет и отображение актуальных рыночных показателей.
- Разработать механизм поиска, фильтрации и сортировки криптовалют: Реализовать отбор монет по цене, рыночной капитализации, объему торгов и изменению цены, а также поддержку серверной сортировки и пагинации.
- Реализовать модуль аналитики и статистики: Обеспечить сравнение монет, построение интерактивных графиков и расчет пользовательской статистики с различными периодами агрегации (часы, дни, недели).
- Реализовать хранение данных в MongoDB: Спроектировать и внедрить структуру хранения данных о пользователях, метаданных монет, исторических снимках котировок и пользовательских пресетах статистики с необходимыми индексами.
- Разработать модуль импорта/экспорта данных: Реализовать административные операции массового экспорта и импорта данных приложения с валидацией входных файлов и контролем целостности данных.

- Создать пользовательский интерфейс и обеспечить удобную навигацию: Разработать адаптивный веб-интерфейс для просмотра списков монет, деталей, истории и результатов аналитики, а также обеспечить понятную навигацию между ключевыми разделами приложения.
- Интегрировать внешние источники рыночных данных: Обеспечить синхронизацию с CoinMarketCap API для обновления котировок и поддержку режима работы на локальных данных без внешних зависимостей.

1.3. Предлагаемое решение

В рамках проекта реализован специализированный веб-сервис для хранения, представления и анализа данных по криптовалютам, обеспечивающий работу с пользовательским портфелем, историческими котировками, фильтрацией, сортировкой и интерактивной аналитикой. В качестве основы для хранения данных (информации о пользователях, ролях доступа, метаданных монет, исторических снимках цен, watchlist, избранном и пользовательских наборах статистики) используется нереляционная база данных MongoDB. Это позволяет эффективно управлять как оперативными рыночными данными, так и персонализированными аналитическими представлениями, а также поддерживать административные сценарии импорта/экспорта и синхронизацию с внешним API CoinMarketCap.

1.4. Качественные требования к решению

В рамках разработки сервиса для хранения, представления и анализа данных по криптовалютам особое внимание уделяется качественным характеристикам системы, обеспечивающим её надежную работу, удобство использования для пользователей и администраторов, а также готовность к дальнейшему развитию функционала. Ниже перечислены ключевые требования, которым соответствует реализованное решение:

- Надёжность хранения рыночных и пользовательских данных: Система обеспечивает целостность данных о пользователях, watchlist, избранном, исторических снимках котировок и статистических пресетах при операциях обновления, импорта/экспорта и аналитической обработки.
- Интуитивно понятный пользовательский интерфейс (UI) и опыт взаимодействия (UX): Функции регистрации, авторизации, просмотра портфеля, поиска/фильтрации монет, анализа графиков и управления персональными настройками логично организованы и удобны для повседневного использования.
- Эффективная и интерактивная визуализация аналитики: Динамика цен и результаты статистических расчетов наглядно отображаются в графическом виде, поддерживают пользовательские сценарии сравнения и позволяют быстро интерпретировать изменения рынка.
- Гибкость и масштабируемость архитектуры: Решение спроектировано с учетом расширения аналитических сценариев, добавления новых источников данных, усложнения вычислительных модулей, а также роста числа пользователей и объемов исторических данных.
- Контроль и управляемость для администраторов: Административные функции предоставляют инструменты для контроля импорта/экспорта, мониторинга состояния данных и управления доступом, что обеспечивает безопасную эксплуатацию системы.

СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ

2.1. Макет UI

(https://github.com/moevm/nsq1h26-crypto/blob/main/ui_mockup.jpg)

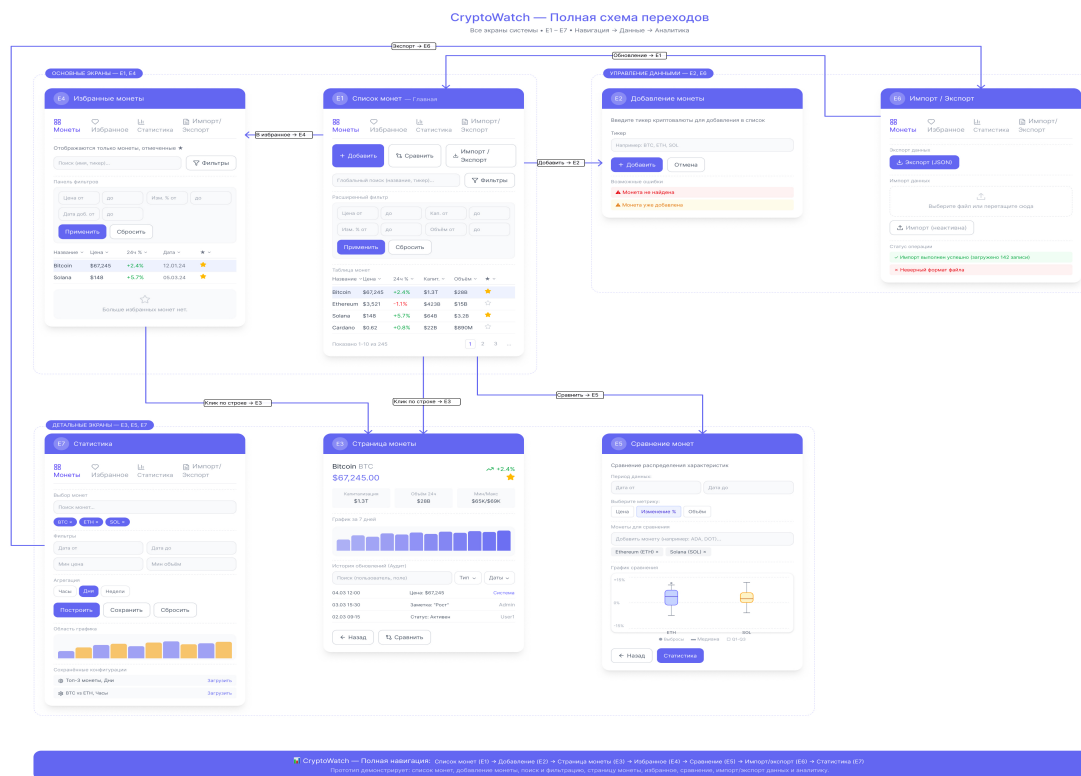


Рисунок 1 – Макет UI

2.2. Сценарии использования

Действующие лица

Действующее лицо	Описание
Гость	Неавторизованный посетитель веб-приложения. Может зарегистрироваться или войти в систему
Пользователь	Авторизованный пользователь веб-приложения с ролью "пользователь". Просматривает котировки, управляет персональным списком отслеживаемых монет, списком избранных монет и своими наборами статистики
Администратор	Авторизованный пользователь с ролью "администратор". Имеет все возможности обычного пользователя, а также может выполнять массовый импорт и экспорт данных
Система	Веб-приложение CryptoWatch: фронтенд (Next.js) + REST API (Spring Boot, Java) + MongoDB

Перечень сценариев

Просмотр и поиск

ID	Название	Связь
UC-1	Просмотр списка монет	--
UC-2	Добавление монеты в список отслеживаемых	--
UC-3	Поиск и фильтрация монет	Расширение UC-1
UC-4	Просмотр страницы монеты	--
UC-5	Сравнение двух монет	Расширение UC-4

Управление избранным

ID	Название	Связь
UC-6	Просмотр избранных монет	--
UC-7	Добавление монеты в избранное	--
UC-8	Удаление монеты из избранного	Альтернатива от UC-6

Статистика и аналитика

ID	Название	Связь
UC-9	Ручное обновление данных	--
UC-10	Импорт и экспорт данных	--
UC-11	Настройка и сохранение кастомной статистики	--

Авторизация и управление доступом

ID	Название	Связь
UC-12	Регистрация	--
UC-13	Логин	--
UC-14	Выход	--

Сценарии в рамках прототипа "хранение и представление"

ID	Название
UC-1	Просмотр списка монет
UC-2	Добавление монеты в список отслеживаемых
UC-3	Поиск и фильтрация монет
UC-4	Просмотр страницы монеты
UC-6	Просмотр избранных монет
UC-7	Добавление монеты в избранное
UC-8	Удаление монеты из избранного
UC-10	Импорт и экспорт данных
UC-13	Логин
UC-14	Выход

Просмотр и поиск

UC-1. Просмотр списка монет

Действующие лица: пользователь, администратор.

Цель: посмотреть актуальные котировки криптовалют из персонального списка отслеживаемых.

Предусловие: пользователь авторизован.

Результат: пользователь видит таблицу монет из своего персонального списка отслеживаемых с актуальными данными.

Переход к юзкейсу: успешный вход в систему (UC-13) или открытие главной страницы приложения при активной сессии.

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
1	Пользователь / Администратор	Открывает приложение или переходит на главную страницу	Сессия истекла или отсутствует -- переход к UC-13
2	Система	Определяет текущего пользователя и загружает из БД его персональный список отслеживаемых монет	Список отслеживаемых монет пуст -- отображается сообщение " Список отслеживаемых монет пуст. Нажмите кнопку Добавить монету "
3	Система	Загружает актуальные данные по монетам из персонального списка отслеживаемых и отображает таблицу. Столбцы: Название/Тикер, Цена (USD), Изменение за 24ч (%), Капитализация, Объём торгов за 24ч, Избранное	Данных по части монет пока нет -- отображаются только доступные данные; для недоступных позиций показывается статус " Данные пока недоступны "
4	Пользователь / Администратор	Просматривает таблицу	Нажимает на строку монеты -- переход к UC-4
5	Пользователь / Администратор	Использует панель фильтров для поиска и сортировки	Переход к UC-3
6	Пользователь / Администратор	Нажимает кнопку " Избранное " рядом с монетой	Монета не в избранном -- UC-7; монета уже в избранном -- UC-8
7	Пользователь / Администратор	Переходит на вкладку " Избранное " в навигации	Переход к UC-6
8	Пользователь /	Нажимает кнопку " Добавить монету "	Переход к UC-2

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
	Администратор		
9	Пользователь / Администратор	Нажимает кнопку "Обновить"	Переход к УС-9
10	Администратор	Нажимает кнопку "Импорт / Экспорт"	Кнопка доступна только администратору -- переход к УС-10
11	Пользователь / Администратор	Переходит на вкладку "Статистика" в навигации	Переход к УС-11
12	Пользователь / Администратор	Нажимает кнопку "Выйти"	Переход к УС-14

УС-2. Добавление монеты в список отслеживаемых

Действующие лица: пользователь, администратор.

Цель: добавить новую монету в персональный список отслеживаемых.

Предусловие: пользователь авторизован; монета отсутствует в его текущем персональном списке отслеживаемых.

Результат: монета добавлена в персональный список отслеживаемых пользователя в БД; если по монете ещё нет данных в системе, они начинают собираться при следующем обновлении.

Переход к юзкейсу: с главной страницы по нажатию **"Добавить монету"**.

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
1	Пользователь / Администратор	Нажимает кнопку "Добавить монету"	--

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
	ор		
2	Система	Открывает форму с полем "Тикер" и кнопками "Добавить" и "Отмена"	--
3	Пользователь / Администратор	Вводит тикер монеты (например, BTC), нажимает "Добавить"	Нажимает "Отмена" -- форма закрывается
4	Система	Проверяет тикер через CoinMarketCap API или по уже известным монетам в системе	Тикер не найден -- "Монета с таким тикером не найдена"
5	Система	Добавляет тикер в персональный список отслеживаемых текущего пользователя в MongoDB. Показывает уведомление: "[Тикер] добавлен в список отслеживаемых". Монета появляется в таблице пользователя	Монета уже есть в списке отслеживаемых этого пользователя -- "Эта монета уже отслеживается"
6	Система	Если в системе ещё нет данных по монете, помечает её для последующего получения при обновлении котировок	--

UC-3. Поиск и фильтрация монет

Действующие лица: пользователь, администратор.

Цель: найти нужную монету или отфильтровать список по заданным критериям.

Предусловие: пользователь авторизован.

Результат: таблица монет из текущего списка пользователя отфильтрована по заданным параметрам.

Переход к юзкейсу: с главной страницы (UC-1).

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
1	Пользователь	Заполняет любое сочетание	Не заполняет ни одного

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
	Администратор	полей панели фильтров: - "Название / Тикер" -- текстовое поле поиска по подстроке; - "Цена (USD) от" и "до" -- диапазон цены; - "Капитализация от" и "до" -- диапазон капитализации; - "Изменение за 24ч (%)" и "до" -- диапазон изменения цены; - "Объём торгов за 24ч от" и "до" -- диапазон объёма.	поля -- таблица отображает все монеты из списка отслеживаемых без ограничений
2	Пользователь / Администратор	Нажимает "Применить"	Нажимает "Сбросить" -- все поля очищаются, таблица возвращается к исходному виду
3	Система	Обновляет таблицу по заданным критериям	Совпадений нет -- "Монеты не найдены"
4	Пользователь / Администратор	Нажимает на заголовок нужного столбца для сортировки (Цена, Капитализация, Изменение за 24ч)	Не использует сортировку -- результат остаётся без изменений
5	Система	Сортирует таблицу по выбранному столбцу. При повторном нажатии меняет направление	--
6	Пользователь / Администратор	Просматривает результаты	Нажимает на строку монеты -- UC-4; нажимает кнопку "Избранное" -- UC-7 или UC-8

UC-4. Просмотр страницы монеты

Действующие лица: пользователь, администратор.

Цель: посмотреть подробные данные и динамику цены выбранной монеты.

Предусловие: пользователь авторизован; монета присутствует в системе.

Результат: пользователь ознакомился с детальными данными и графиком цены.

Переход к юзкейсу: нажатие на строку монеты в таблице (UC-1) или на карточку в разделе "Избранное" (UC-6).

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
1	Пользователь / Администратор	Нажимает на строку монеты в таблице	--
2	Система	Открывает страницу монеты с текущей ценой, изменением за 24ч, капитализацией, объёмом торгов, графиком цены за 7 дней, минимальной/максимальной/средней ценой за период	Данных меньше 7 дней -- график строится по доступному диапазону, отображается предупреждение "Данные доступны с [дата]"
3	Пользователь / Администратор	Изучает данные и график	Нажимает "Сравнить" -- UC-5; нажимает "Избранное" -- UC-7 или UC-8
4	Пользователь / Администратор	Заполняет поля фильтра истории обновлений: - "Дата от" и "до" -- диапазон дат; - "Цена от" и "до" -- диапазон цены в записях истории; - "Объём торгов от" и "до" -- диапазон объёма торгов в записях истории.	Не заполняет фильтр -- отображается полная история
5	Пользователь / Администратор	Нажимает "Применить"	Нажимает "Сбросить" -- фильтр очищается, история отображается полностью
6	Система	Обновляет таблицу истории обновлений по заданным критериям	Совпадений нет -- "Записей за выбранный период не найдено"

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
7	Пользователь / Администратор	Нажимает "Назад"	--
8	Система	Возвращает пользователя на предыдущую страницу	--

UC-5. Сравнение двух монет

Действующие лица: пользователь, администратор.

Цель: сравнить динамику цен двух монет на одном графике.

Предусловие: пользователь авторизован.

Результат: пользователь видит совмещённый график цен двух монет.

Переход к юзкейсу: со страницы монеты (UC-4) по нажатию "Сравнить".

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
1	Система	Открывает панель сравнения. Первая монета уже задана. Показывает поле " Выберите вторую монету " с поиском по названию/тикеру	--
2	Пользователь / Администратор	Выбирает вторую монету из выпадающего списка	Выбирает ту же монету -- " Выберите другую монету для сравнения "
3	Пользователь / Администратор	Задаёт диапазон дат: " Дата от " и " до "	Не заполняет -- используется диапазон по умолчанию (последние 7 дней)
4	Система	Показывает переключатель типа графика: " Линейный " или " Box plot "	--

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
5	Пользователь / Администратор	Выбирает тип графика	--
6	Система	Строит совмещённый график за выбранный период. Линейный: две линии разных цветов, ось Y в процентах от начала периода. Box plot: ящик с усами -- показывает медиану, квартили и выбросы цены за период	Недостаточно данных по одной из монет -- "Недостаточно данных для [тикер]"
7	Пользователь / Администратор	Изучает график	Нажимает "Назад" -- возврат на страницу первой монеты

Управление избранным

УС-6. Просмотр избранных монет

Действующие лица: пользователь, администратор.

Цель: быстро проверить состояние монет из своего списка избранного.

Предусловие: пользователь авторизован; список избранных монет не пуст.

Результат: пользователь видит актуальные данные по избранным монетам, сохранённым в БД за его учётной записью.

Переход к юзкейсу: нажатие на вкладку "Избранное" в навигации.

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
1	Пользователь / Администратор	Переходит на вкладку "Избранное"	--
2	Система	Считывает список избранных монет текущего пользователя из	Список пуст -- "Вы ещё не добавили ни"

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
		MongoDB и отображает таблицу только для избранных монет (те же столбцы, что в UC-1)	одной монеты. Нажмите кнопку Избранное рядом с монетой в общем списке"
3	Пользователь / Администратор	Заполняет любое сочетание полей панели фильтров: - "Название / Тикер" -- текстовое поле поиска по подстроке; - "Цена (USD) от" и "до" -- диапазон цены; - "Капитализация от" и "до" -- диапазон капитализации; - "Изменение за 24ч (%)" от" и "до" -- диапазон изменения цены; - "Объём торгов за 24ч от" и "до" -- диапазон объёма.	Не заполняет ни одного поля -- таблица отображает все избранные монеты без ограничений
4	Пользователь / Администратор	Нажимает "Применить"	Нажимает "Сбросить" -- все поля очищаются, таблица возвращается к исходному виду
5	Система	Обновляет таблицу по заданным критериям	Совпадений нет -- "Монеты не найдены"
6	Пользователь / Администратор	Нажимает на заголовок нужного столбца для сортировки (Цена, Капитализация, Изменение за 24ч)	Не использует сортировку -- результат остаётся без изменений
7	Система	Сортирует таблицу по выбранному столбцу. При повторном нажатии меняет направление	--
8	Пользователь / Администратор	Просматривает список	Нажимает на строку монеты -- UC-4; нажимает кнопку "Избранное" -- UC-8

УС-7. Добавление монеты в избранное

Действующие лица: пользователь, администратор.

Цель: добавить монету в персональный список избранного для быстрого доступа.

Предусловие: пользователь авторизован; монета не находится в списке избранных текущего пользователя.

Результат: монета добавлена в список избранных текущего пользователя в БД.

Переход к юзкейсу: нажатие кнопки "Избранное" в таблице (УС-1) или на странице монеты (УС-4).

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
1	Пользователь / Администратор	Нажимает кнопку "Избранное" рядом с монетой	--
2	Система	Добавляет тикер монеты в список избранных текущего пользователя в MongoDB. Кнопка "Избранное" становится активной. Показывает уведомление: "[Тикер] добавлен в избранное"	Сессия истекла -- переход к УС-13

УС-8. Удаление монеты из избранного

Действующие лица: пользователь, администратор.

Цель: убрать монету из персонального списка избранного.

Предусловие: пользователь авторизован; монета находится в списке избранных текущего пользователя.

Результат: монета удалена из списка избранных текущего пользователя в БД.

Переход к юзкейсу: нажатие активной кнопки "Избранное" в разделе "Избранное" (UC-6) или на странице монеты (UC-4).

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
1	Пользователь / Администратор	Нажимает активную кнопку "Избранное" рядом с монетой	--
2	Система	Удаляет тикер монеты из списка избранных текущего пользователя в MongoDB. Кнопка "Избранное" становится неактивной. Показывает уведомление: "[Тикер] удалён из избранного"	Сессия истекла -- переход к UC-13
3	Система	Если удаление выполнено из раздела "Избранное" -- строка монеты исчезает из таблицы	--

Статистика и аналитика

UC-9. Ручное обновление данных

Действующие лица: пользователь, администратор.

Цель: принудительно обновить котировки, не дожидаясь следующего запуска по расписанию.

Предусловие: пользователь авторизован.

Результат: данные в таблице обновлены до актуальных котировок.

Переход к юзкейсу: с главной страницы (UC-1).

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
1	Пользователь / Администратор	Нажимает кнопку "Обновить"	--
2	Система	Блокирует кнопку,	--

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
		показывает индикатор загрузки, запрашивает данные у CoinMarketCap API и сохраняет снимки в MongoDB	
3	Система	Обновляет таблицу, разблокирует кнопку, отображает метку "Обновлено: [дата и время]"	Запрос к API завершился ошибкой -- "Не удалось получить данные. Попробуйте позже" ; таблица остаётся с прежними данными

UC-10. Импорт и экспорт данных

Действующие лица: администратор.

Цель: выгрузить все данные приложения в файл или загрузить данные из файла.

Предусловие: пользователь авторизован и имеет роль "администратор".

Результат: данные экспортированы в файл или импортированы в БД.

Переход к юзкейсу: с главной страницы по нажатию "Импорт / Экспорт".

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
1	Администратор	Нажимает кнопку "Импорт / Экспорт"	Обычный пользователь пытается открыть раздел напрямую -- "Недостаточно прав для выполнения операции"
2	Система	Проверяет роль пользователя и открывает страницу с кнопками "Экспорт", "Выбрать файл" и неактивной кнопкой "Импорт"	--
3	Администратор	Нажимает "Экспорт"	Переходит к шагу 5 для

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
	атор		импорта
4	Система	Формирует файл со всеми данными приложения в машиночитаемом формате и начинает его скачивание	--
5	Администратор	Нажимает " Выбрать файл ", выбирает файл в диалоге	--
6	Система	Активирует кнопку " Импорт "	Формат файла не поддерживается -- " Неверный формат файла ", кнопка " Импорт " остаётся неактивной
7	Администратор	Нажимает " Импорт "	--
8	Система	Заменяет содержимое БД данными из файла. Показывает уведомление: " Импорт выполнен успешно. Загружено записей: [N] "	Файл повреждён или содержит некорректные данные -- " Ошибка импорта. Проверьте файл "

УС-11. Настройка и сохранение кастомной статистики

Действующие лица: пользователь, администратор.

Цель: построить кастомный график по выбранным монетам с нужными параметрами и сохранить настройки для повторного использования.

Предусловие: пользователь авторизован.

Результат: построен кастомный график; один или несколько наборов настроек статистики пользователя сохранены в БД и доступны при следующем открытии.

Переход к юзкейсу: переход на вкладку "Статистика" в навигации.

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
1	Пользователь /	Переходит на вкладку "Статистика"	--

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
	Администратор		
2	Система	Открывает страницу статистики. Если у пользователя есть сохранённые наборы настроек -- отображает их список и последний использованный набор настроек	Сохранённых настроек нет -- отображается пустая форма настройки
3	Пользователь / Администратор	Выбирает существующий набор настроек или нажимает "Новый набор"	--
4	Пользователь / Администратор	Выбирает одну или несколько валютных пар из выпадающего списка с поиском по тикеру	--
5	Пользователь / Администратор	Задаёт фильтры: временной диапазон, минимальная/максимальная цена, минимальный объём торгов	Не заполняет фильтры -- данные не ограничиваются
6	Пользователь / Администратор	Выбирает правило агрегации: "По часам" , "По дням" или "По неделям"	--
7	Пользователь / Администратор	Нажимает "Построить"	Выбранных монет в БД нет за указанный период -- "Нет данных для отображения по заданным параметрам"
8	Система	Строит график согласно выбранным парам, фильтрам и правилу агрегации	--
9	Пользователь / Администратор	Нажимает "Сохранить настройки"	Нажимает "Сбросить" -- все поля очищаются, график скрывается
10	Система	Запрашивает имя набора настроек и режим сохранения: создать новый или перезаписать существующий	--

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
11	Пользователь / Администратор	Вводит имя набора и подтверждает сохранение	Имя не введено -- "Укажите название набора настроек"
12	Система	Сохраняет текущие фильтры, правило агрегации и набор валютных пар в MongoDB как отдельный пользовательский набор настроек, связанный с идентификатором пользователя. Показывает уведомление: "Настройки сохранены"	--

Авторизация и управление доступом

UC-12. Регистрация

Действующие лица: гость.

Цель: создать новую учётную запись для работы с персональными данными приложения.

Предусловие: пользователь не авторизован.

Результат: создана новая учётная запись с уникальным идентификатором пользователя, логином, паролем, хранимым в виде hash + salt, ролью "пользователь", пустым списком отслеживаемых монет, пустым списком избранных монет и пустым набором сохранённых настроек статистики.

Переход к юзкейсу: со страницы логина по нажатию "Регистрация".

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
1	Гость	Открывает страницу регистрации	--
2	Система	Отображает форму с полями "Логин", "Пароль",	--

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
		"Подтверждение пароля" и кнопками "Зарегистрироваться" и "Отмена"	
3	Гость	Заполняет форму и нажимает "Зарегистрироваться"	Нажимает "Отмена" -- возврат на страницу логина
4	Система	Проверяет корректность введённых данных и уникальность логина	Логин уже занят -- "Пользователь с таким логином уже существует"; пароль и подтверждение не совпадают -- "Пароли не совпадают"; обязательные поля не заполнены -- "Заполните обязательные поля"
5	Система	Генерирует уникальный идентификатор пользователя, создаёт salt , вычисляет hash пароля и сохраняет нового пользователя в MongoDB с ролью "пользователь"	Ошибка сохранения -- "Не удалось создать пользователя. Попробуйте позже"
6	Система	Показывает уведомление "Регистрация выполнена успешно" и предлагает перейти к логину	Переход к UC-13

UC-13. Логин

Действующие лица: гость.

Цель: войти в приложение под своей учётной записью и получить доступ к персональным данным.

Предусловие: пользователь зарегистрирован и не авторизован.

Результат: пользователь авторизован; система определила его роль и загрузила персональные данные из БД.

Переход к юзкейсу: открытие приложения без активной сессии или переход со страницы регистрации (UC-12).

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
1	Гость	Открывает приложение	Активной сессии нет -- открывается форма логина
2	Система	Отображает форму с полями "Логин" и "Пароль", кнопками "Войти" и "Регистрация"	--
3	Гость	Вводит логин и пароль, нажимает "Войти"	Нажимает "Регистрация" -- переход к UC-12
4	Система	Находит пользователя по логину, вычисляет хэш введенного пароля с сохранённым salt и сравнивает его с сохранённым значением	Логин не найден или пароль неверный -- "Неверный логин или пароль"
5	Система	Создаёт авторизованную сессию, определяет роль пользователя ("пользователь" / "администратор") и загружает его персональные данные (список отслеживаемых монет, список избранных монет, сохранённые настройки статистики)	Ошибка авторизации -- "Не удалось выполнить вход. Попробуйте позже"
6	Система	Перенаправляет пользователя на главную страницу приложения	Переход к UC-1

UC-14. Выход

Действующие лица: пользователь, администратор.

Цель: завершить текущую пользовательскую сессию.

Предусловие: пользователь авторизован.

Результат: сессия завершена, доступ к персональным данным закрыт.

Переход к юзкейсу: нажатие кнопки "Выйти" в интерфейсе приложения.

№ шага	Действующее лицо	Шаг / Действие	Альтернативный сценарий
1	Пользователь / Администратор	Нажимает кнопку "Выйти"	--
2	Система	Завершает пользовательскую сессию и удаляет данные авторизации на клиенте	Ошибка завершения сессии -- "Не удалось завершить сеанс. Попробуйте ещё раз"
3	Система	Перенаправляет пользователя на страницу логина	Переход к UC-13

3. МОДЕЛЬ ДАННЫХ

3.1 Нереляционная модель

3.1.1 Графическое представление

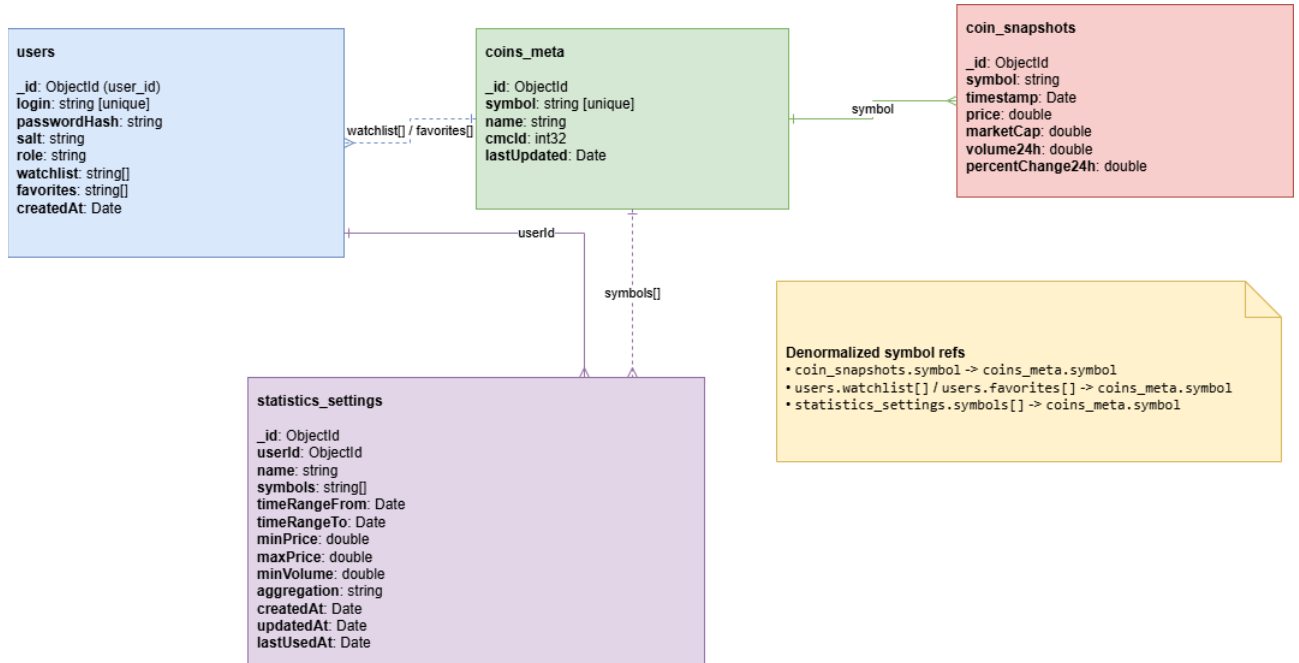


Рисунок 2 – Графическое представление нереляционной модели

3.1.2 Описание коллекций

Размеры типов данных (BSON, MongoDB):

- ObjectId: 12
- string(L): 4 (префикс длины) + L
- double: 8
- Date: 8
- int32: 4
- string[](N, L): 4 (длина массива) + N * (4 + L)

users (учетные записи и персональные данные пользователей):

- _id: ObjectId -- уникальный идентификатор пользователя (user_id)
- login: string -- логин пользователя <<unique>>

- passwordHash: string -- хэш пароля
- salt: string -- соль для вычисления хэша
- role: string -- роль пользователя (user, admin)
- watchlist: string[] -- список отслеживаемых монет пользователя
- favorites: string[] -- список избранных монет пользователя
- createdAt: Date -- дата создания учетной записи

coins_meta (хранение метаданных отслеживаемых монет):

- _id: ObjectId
- symbol: string -- тикер монеты, уникальный идентификатор (например, BTC)
- name: string -- полное название монеты (например, Bitcoin)
- cmcId: int32 -- идентификатор монеты в CoinMarketCap
- lastUpdated: Date -- дата и время последнего обновления данных

coin_snapshots (хранение снимков цен -- временные ряды):

- _id: ObjectId
- symbol: string -- тикер монеты [-> coins_meta]
- timestamp: Date -- дата и время снимка
- price: double -- цена в USD
- marketCap: double -- рыночная капитализация в USD
- volume24h: double -- объём торгов за 24 часа в USD
- percentChange24h: double -- изменение цены за 24 часа в процентах

statistics_settings (хранение сохраненных настроек кастомной статистики пользователя):

- _id: ObjectId
- userId: ObjectId -- идентификатор владельца [-> users]

- name: string -- название набора настроек
- symbols: string[] -- список тикеров выбранных монет
- timeRangeFrom: Date -- начало временного диапазона
- timeRangeTo: Date -- конец временного диапазона
- minPrice: double -- минимальная цена (фильтр)
- maxPrice: double -- максимальная цена (фильтр)
- minVolume: double -- минимальный объём торгов (фильтр)
- aggregation: string -- правило агрегации (hours, days, weeks)
- createdAt: Date -- дата создания набора
- updatedAt: Date -- дата последнего сохранения / перезаписи
- lastUsedAt: Date -- дата последнего применения набора

Ключевые индексы модели:

- users.login -- уникальный индекс для регистрации и логина
- coins_meta.symbol -- уникальный индекс по тикеру
- coin_snapshots(symbol, timestamp) -- составной индекс для истории и получения актуального снимка
- statistics_settings(userId, lastUsedAt) -- индекс для загрузки пользовательских наборов статистики
- statistics_settings(userId, name) -- уникальный индекс имени набора в рамках пользователя

Оценка объема информации

Примем следующие константы:

- AvgSymbolLength: 4 (BTC, ETH, SOL и т.д.)
- AvgCoinNameLength: 8 (Bitcoin, Ethereum и т.д.)
- AvgUserLoginLength: 12
- AvgPasswordHashLength: 64
- AvgSaltLength: 24

- AvgRoleLength: 5 (user, admin)
- AvgWatchlistSize: 10
- AvgFavoritesSize: 5
- AvgSettingsNameLength: 16
- AvgAggregationLength: 5 (hours, days, weeks)
- AvgSymbolsPerStats: 3
- MongoDocOverhead: 50 (BSON-заголовок документа + байты типов полей + байты имён полей)

Получим следующие формулы для размера одного документа:

- users: $12 + (4+12) + (4+64) + (4+24) + (4+5) + (4 + 10*(4+4)) + (4 + 5*(4+4)) + 8 + \text{MongoDocOverhead} = 269 + 50 = 319$
- coins_meta: $12 + (4+4) + (4+8) + 4 + 8 + \text{MongoDocOverhead} = 44 + 50 = 94$
- coin_snapshots: $12 + (4+4) + 8 + 8 + 8 + 8 + 8 + 8 + \text{MongoDocOverhead} = 60 + 50 = 110$
- statistics_settings: $12 + 12 + (4+16) + (4 + 3*(4+4)) + 8 + 8 + 8 + 8 + 8 + (4+5) + 8 + 8 + 8 + \text{MongoDocOverhead} = 145 + 50 = 195$

Оценим общий размер БД (U -- количество пользователей, M -- количество уникальных монет, S -- число снимков на монету, R -- общее число сохраненных наборов статистики):

$\text{MongoCollectionOverhead} = 2 * 1024 * 1024 \text{ байт} = 2 \text{ Мбайт на коллекцию}$

$$\begin{aligned}
 V &= \text{MongoCollectionOverhead} * 4 \\
 &+ U * 319 \\
 &+ M * 94 \\
 &+ M * S * 110
 \end{aligned}$$

$$+ R * 195$$

Для пользовательской части модели удобно оценить рост по числу пользователей. Примем:

$$U = X$$

$$M = 50 \quad \text{-- в системе отслеживается 50 уникальных монет}$$

$$S = 672 \quad \text{-- 1 снимок каждые 15 минут за 7 дней}$$

$$R = 3 * X \quad \text{-- в среднем 3 сохраненных набора статистики на пользователя}$$

$$\text{MongoCollectionOverhead} = 2 \text{ Мбайт}$$

Тогда:

$$V(X) = 8.00 \text{ Мбайт}$$

$$+ 50 * 94$$

$$+ 50 * 672 * 110$$

$$+ X * (319 + 3 * 195)$$

$$= 8.00 \text{ Мбайт} + 3\,700\,700 \text{ байт} + X * 904 \text{ байт}$$

$$\sim 11.53 \text{ Мбайт} + X * 0.88 \text{ Кбайт}$$

Избыточность данных

Избыточность модели складывается из:

- Технических накладных расходов MongoDB на коллекцию: по 2 Мбайт на 4 коллекции = 8 Мбайт
- Технических накладных расходов MongoDB на каждый документ: $\text{MongoDocOverhead} = 50 \text{ байт}$
- Избыточности самих данных:

- поле `symbol` дублируется
в `coin_snapshots`, `watchlist`, `favorites` и `statistics_settings`
- имя набора настроек и метки времени
(`createdAt`, `updatedAt`, `lastUsedAt`) хранятся отдельно для каждого
сохраненного набора

Минимальный (сырой) размер хранимой информации без служебных байтов BSON и служебных полей:

```
RawSize(X) = 50 * 24                -- coins_meta: symbol(4) +
name(8) + cmcId(4) + lastUpdated(8)
           + 50 * 672 * 44          -- coin_snapshots: symbol(4)
+ ts(8) + price(8) + mktCap(8) + vol(8) + pct(8)
           + X * 173                -- users: login(12) +
hash(64) + salt(24) + role(5) + 10*symbol(4) + 5*symbol(4) +
createdAt(8)
           + 3 * X * 109            -- statistics_settings:
userId(12) + name(16) + 3*symbol(4) + 2*Date(16) + 3*double(24)
+ aggregation(5) + 3*Date(24)

= 1 479 600 + 500 * X
```

Отношение полезной нагрузки к полному объёму:

$$\text{RawSize}(X) / V(X) =$$

$$= (1\,479\,600 + 500 * X) / (12\,089\,308 + 904 * X)$$

При больших X пользовательская часть модели стремится к коэффициенту полезной нагрузки:

$$500 / 904 \sim 0.55$$

То есть для слоя пользовательских данных около 55% объема занимают полезные данные, остальное - служебные байты BSON и внутренняя структура.

Направление роста модели

Определим направление роста при предыдущих константах:

dV/dU = 904 байт -- добавление одного пользователя с 3 наборами статистики

dV/dM = 74 014 байт -- добавление одной новой монеты с историей за 7 дней

$dV/d(M*S)$ = 110 байт -- добавление одного снимка для одной монеты

dV/dR = 195 байт -- добавление одного сохраненного набора статистики

Рост модели линейный. Основным фактором роста базы остается накопление снимков в `coin_snapshots`. Второй стабильный источник роста - документы пользователей и сохраненные наборы статистики.

Примеры данных

Модель

- Пользователь user с ролью user
- Администратор admin
- Монета BTC (Bitcoin)
- Монета ETH (Ethereum)
- Два снимка для BTC
- Один снимок для ETH
- Сохранённый набор статистики пользователя user для пары BTC/ETH

Представление модели в БД

users

```

{
  "_id": ObjectId("665a1b2c3d4e5f6a7b8c9d10"),
  "login": "user",
  "passwordHash":
"9e7f4cf8a1c29d4a8b0a558d22d37f1c6119d6ef4a72ec3b3c28f0b20ec77
a91",
  "salt": "m4G7c2k9Qp1s8Zx5Vb3Nw0Yt",
  "role": "user",
  "watchlist": ["BTC", "ETH"],
  "favorites": ["BTC"],
  "createdAt": ISODate("2026-03-27T12:00:00Z")
},
{
  "_id": ObjectId("665a1b2c3d4e5f6a7b8c9d11"),
  "login": "admin",
  "passwordHash":
"0a1fb5bbd5c2a447da4f2752ae2f0d7fd8c964bc239be1bc7b1cf3477512e
81a",
  "salt": "Qw8Yp2Ls4Nh7Jk1Vc5Rm9Tz0",
  "role": "admin",
  "watchlist": ["BTC", "ETH", "SOL"],
  "favorites": ["ETH"],
  "createdAt": ISODate("2026-03-27T12:05:00Z")
}

coins_meta
{
  "_id": ObjectId("665a1b2c3d4e5f6a7b8c9d0a"),
  "symbol": "BTC",

```

```

    "name": "Bitcoin",
    "cmcId": 1,
    "lastUpdated": ISODate("2026-03-27T12:00:00Z")
  },
  {
    "_id": ObjectId("665a1b2c3d4e5f6a7b8c9d0b"),
    "symbol": "ETH",
    "name": "Ethereum",
    "cmcId": 1027,
    "lastUpdated": ISODate("2026-03-27T12:00:00Z")
  }

```

coin_snapshots

```

{
  "_id": ObjectId("665a1b2c3d4e5f6a7b8c9d0c"),
  "symbol": "BTC",
  "timestamp": ISODate("2026-03-27T11:45:00Z"),
  "price": 83400.25,
  "marketCap": 1651234567890,
  "volume24h": 43210987654,
  "percentChange24h": 1.35
},
{
  "_id": ObjectId("665a1b2c3d4e5f6a7b8c9d0d"),
  "symbol": "BTC",
  "timestamp": ISODate("2026-03-27T12:00:00Z"),
  "price": 83512.80,
  "marketCap": 1653456789012,
  "volume24h": 44321098765,

```

```

    "percentChange24h": 1.49
  },
  {
    "_id": ObjectId("665a1b2c3d4e5f6a7b8c9d0e"),
    "symbol": "ETH",
    "timestamp": ISODate("2026-03-27T12:00:00Z"),
    "price": 1945.60,
    "marketCap": 234567890123,
    "volume24h": 18765432109,
    "percentChange24h": -0.72
  }

```

statistics_settings

```

{
  "_id": ObjectId("665a1b2c3d4e5f6a7b8c9d0f"),
  "userId": ObjectId("665a1b2c3d4e5f6a7b8c9d10"),
  "name": "BTC vs ETH / week",
  "symbols": ["BTC", "ETH"],
  "timeRangeFrom": ISODate("2026-03-24T00:00:00Z"),
  "timeRangeTo": ISODate("2026-03-29T00:00:00Z"),
  "minPrice": 1000.0,
  "maxPrice": 200000.0,
  "minVolume": 1000000.0,
  "aggregation": "days",
  "createdAt": ISODate("2026-03-29T10:30:00Z"),
  "updatedAt": ISODate("2026-03-29T10:30:00Z"),
  "lastUsedAt": ISODate("2026-03-29T11:55:00Z")
}

```

Примеры запросов

UC-12. Регистрация

// Шаг 1: проверить уникальность логина

```
db.users.findOne({ login: "user" })
```

// Шаг 2: создать пользователя

```
db.users.insertOne({
  login: "user",
  passwordHash:
    "9e7f4cf8a1c29d4a8b0a558d22d37f1c6119d6ef4a72ec3b3c28f0b20ec77a91",
  salt: "m4G7c2k9Qp1s8Zx5Vb3Nw0Yt",
  role: "user",
  watchlist: [],
  favorites: [],
  createdAt: new Date()
})
```

Количество запросов: 2, задействованных коллекций: 1 (users)

UC-13. Логин

// Шаг 1: получить учётную запись по логину

```
const user = db.users.findOne(
  { login: "user" },
  { login: 1, passwordHash: 1, salt: 1, role: 1, watchlist: 1,
    favorites: 1 }
)
```

// Шаг 2: загрузить сохраненные наборы статистики пользователя


```
db.statistics_settings
  .find({ userId: user._id })
  .sort({ lastUsedAt: -1, updatedAt: -1 })
```

Количество запросов: 2, задействованных
коллекций: 2 (users, statistics_settings)

UC-1. Просмотр списка монет

// Шаг 1: получить watchlist пользователя

```
const user = db.users.findOne(
  { _id: ObjectId("665a1b2c3d4e5f6a7b8c9d10") },
  { watchlist: 1, favorites: 1 }
)
```

// Шаг 2: получить последние снимки только по монетам из watchlist

```
db.coin_snapshots.aggregate([
  { $match: { symbol: { $in: user.watchlist } } },
  { $sort: { symbol: 1, timestamp: -1 } },
  { $group: { _id: "$symbol", doc: { $first: "$$ROOT" } } },
  { $replaceRoot: { newRoot: "$doc" } },
  { $sort: { marketCap: -1 } }
])
```

Количество запросов: 2, задействованных
коллекций: 2 (users, coin_snapshots)

UC-2. Добавление монеты в список отслеживаемых

// Шаг 1: проверить, существует ли монета в справочнике

```
db.coins_meta.findOne({ symbol: "BTC" })
```

```
// Шаг 2: добавить тикер в watchlist текущего пользователя
db.users.updateOne(
  { _id: ObjectId("665a1b2c3d4e5f6a7b8c9d10") },
  { $addToSet: { watchlist: "BTC" } }
)
```

Количество запросов: 2, задействованных коллекций: 2 (coins_meta, users). Если монеты еще нет в coins_meta, появляется еще один запрос на её вставку.

UC-7. Добавление монеты в избранное

```
db.users.updateOne(
  { _id: ObjectId("665a1b2c3d4e5f6a7b8c9d10") },
  { $addToSet: { favorites: "BTC" } }
)
```

Количество запросов: 1, задействованных коллекций: 1 (users)

UC-8. Удаление монеты из избранного

```
db.users.updateOne(
  { _id: ObjectId("665a1b2c3d4e5f6a7b8c9d10") },
  { $pull: { favorites: "BTC" } }
)
```

Количество запросов: 1, задействованных коллекций: 1 (users)

UC-10. Импорт и экспорт данных

Экспорт всех данных

```
const usersData = await db.users.find({}).toArray()
```

```

const coinsData = await db.coins_meta.find({}).toArray()
const          snapshotsData          =          await
db.coin_snapshots.find({}).toArray()
const          statsData              =          await
db.statistics_settings.find({}).toArray()
// Сериализация в JSON и отправка клиенту

```

Количество запросов: 4, задействованных коллекций: 4

Импорт всех данных

```

await db.users.deleteMany({})
await db.coins_meta.deleteMany({})
await db.coin_snapshots.deleteMany({})
await db.statistics_settings.deleteMany({})

await db.users.insertMany(importData.users)
await db.coins_meta.insertMany(importData.coins_meta)
await db.coin_snapshots.insertMany(importData.coin_snapshots)
await
db.statistics_settings.insertMany(importData.statistics_settin
gs)

```

Количество запросов: 8, задействованных коллекций: 4

УС-11. Настройка и сохранение кастомной статистики

// Шаг 1: построить агрегированный график

```

db.coin_snapshots.aggregate([
  {
    $match: {
      symbol: { $in: ["BTC", "ETH"] },

```

```

        timestamp: { $gte: ISODate("2026-03-24"), $lte:
ISODate("2026-03-29") },
        price: { $gte: 1000, $lte: 200000 },
        volume24h: { $gte: 1000000 }
    }
},
{
    $group: {
        _id: {
            symbol: "$symbol",
            period: { $dateTrunc: { date: "$timestamp", unit:
"day" } }
        },
        avgPrice: { $avg: "$price" },
        avgVolume: { $avg: "$volume24h" }
    }
},
{ $sort: { "_id.period": 1, "_id.symbol": 1 } }
])

```

// Шаг 2: сохранить или перезаписать пользовательский набор

```

db.statistics_settings.updateOne(
    {
        userId: ObjectId("665a1b2c3d4e5f6a7b8c9d10"),
        name: "BTC vs ETH / week"
    },
    {
        $set: {
            symbols: ["BTC", "ETH"],

```

```

    timeRangeFrom: ISODate("2026-03-24"),
    timeRangeTo: ISODate("2026-03-29"),
    minPrice: 1000.0,
    maxPrice: 200000.0,
    minVolume: 1000000.0,
    aggregation: "days",
    updatedAt: new Date(),
    lastUsedAt: new Date()
  },
  $setOnInsert: {
    createdAt: new Date()
  }
},
{ upsert: true }
)

```

Количество запросов: 2, задействованных коллекций: 2 (coin_snapshots, statistics_settings)

UC-11. Список валют из статистики

```

// Получить список всех тикеров, которые пользователь
использовал в сохраненных наборах статистики
db.statistics_settings.aggregate([
  { $match: { userId: ObjectId("665a1b2c3d4e5f6a7b8c9d10") } },
  { $unwind: "$symbols" },
  { $group: { _id: "$symbols" } },
  { $sort: { _id: 1 } }
])

```

Количество запросов: 1, задействованных
коллекций: 1 (statistics_settings)

UC-11. Максимальное изменение курса

// Получить валюты с наибольшим изменением курса по модулю за
выбранный период

```
db.coin_snapshots.aggregate([
  {
    $match: {
      timestamp: { $gte: ISODate("2026-03-24"), $lt:
ISODate("2026-03-29") }
    }
  },
  { $sort: { symbol: 1, timestamp: 1 } },
  {
    $group: {
      _id: "$symbol",
      firstPrice: { $first: "$price" },
      lastPrice: { $last: "$price" }
    }
  },
  {
    $project: {
      _id: 0,
      symbol: "$_id",
      firstPrice: 1,
      lastPrice: 1,
      changePercent: {
        $multiply: [
```

```

    {
      $divide: [
        { $subtract: ["$lastPrice", "$firstPrice"] },
        "$firstPrice"
      ]
    },
    100
  ]
}
},
{ $addFields: { absChangePercent: { $abs: "$changePercent" } } },
{ $sort: { absChangePercent: -1, symbol: 1 } },
{ $limit: 10 }
])

```

Количество запросов: 1, задействованных коллекций: 1 (coin_snapshots)

3.2. Реляционная модель

3.2.1. Графическое представление

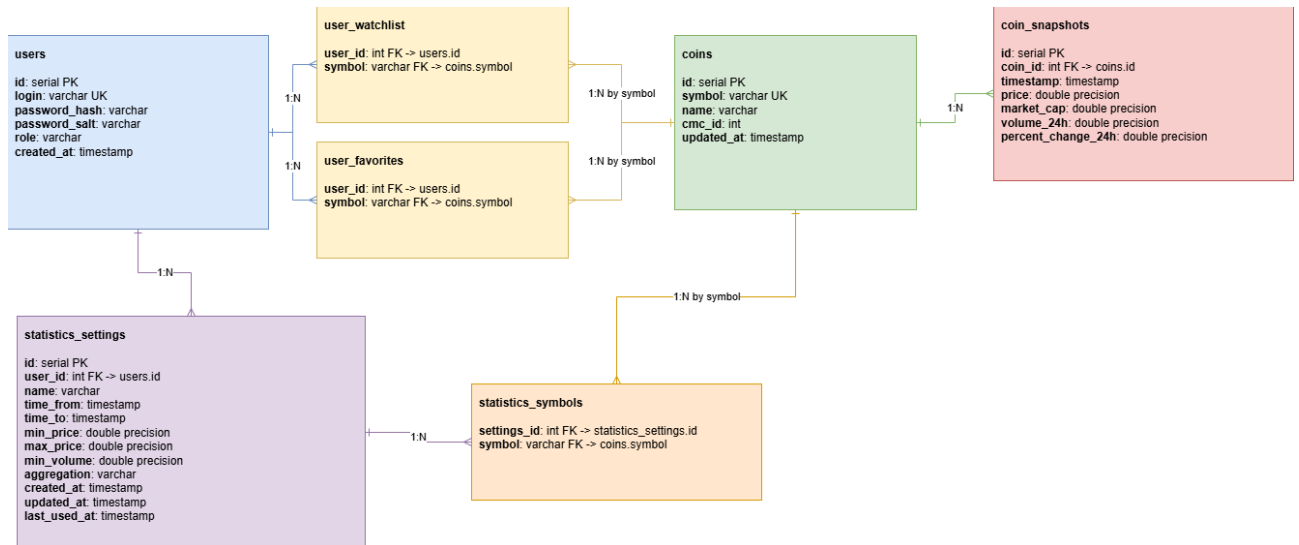


Рисунок 3 – графическое представление реляционной модели

3.2.2. Описание таблиц

Размеры типов данных (PostgreSQL):

- serial / int4: 4
- varchar(N) (хранит фактическую длину): L
- double precision: 8
- timestamp: 8

users (учетные записи пользователей):

- id: serial -- первичный ключ
- login: varchar(64) -- логин <<unique>>
- password_hash: varchar(128) -- хэш пароля
- password_salt: varchar(64) -- соль
- role: varchar(10) -- роль (user, admin)
- created_at: timestamp -- дата создания

user_watchlist (персональный список отслеживаемых монет):

- user_id: int4 -- внешний ключ [-> users.id]

- `symbol: varchar(10)` -- тикер монеты [-> `coins.symbol`]

user_favorites (персональный список избранных монет):

- `user_id: int4` -- внешний ключ [-> `users.id`]
- `symbol: varchar(10)` -- тикер монеты [-> `coins.symbol`]

coins (справочник монет):

- `id: serial` -- первичный ключ
- `symbol: varchar(10)` -- тикер <<unique>>
- `name: varchar(100)` -- полное название
- `cmc_id: int4` -- идентификатор CoinMarketCap
- `updated_at: timestamp` -- дата последнего обновления

coin_snapshots (исторические снимки цен):

- `id: serial` -- первичный ключ
- `coin_id: int4` -- внешний ключ [-> `coins.id`]
- `timestamp: timestamp` -- дата и время снимка
- `price: double precision` -- цена в USD
- `market_cap: double precision` -- рыночная капитализация в USD
- `volume_24h: double precision` -- объём торгов за 24ч в USD
- `percent_change_24h: double precision` -- изменение цены за 24ч

statistics_settings (набор настроек статистики пользователя):

- `id: serial` -- первичный ключ
- `user_id: int4` -- внешний ключ [-> `users.id`]
- `name: varchar(100)` -- название набора
- `time_from: timestamp` -- начало диапазона
- `time_to: timestamp` -- конец диапазона
- `min_price: double precision` -- фильтр минимальной цены

- max_price: double precision -- фильтр максимальной цены
- min_volume: double precision -- фильтр минимального объёма
- aggregation: varchar(10) -- правило агрегации (hours, days, weeks)
- created_at: timestamp
- updated_at: timestamp
- last_used_at: timestamp

statistics_symbols (связь "набор статистики -- монета"):

- settings_id: int4 -- внешний ключ [-> statistics_settings.id]
- symbol: varchar(10) -- тикер монеты [-> coins.symbol]

Ключевые индексы модели:

- users(login) -- уникальный индекс
- coins(symbol) -- уникальный индекс
- coin_snapshots(coin_id, timestamp) -- составной индекс
- user_watchlist(user_id, symbol) -- уникальный индекс
- user_favorites(user_id, symbol) -- уникальный индекс
- statistics_settings(user_id, last_used_at)
- statistics_settings(user_id, name) -- уникальный индекс
- statistics_symbols(settings_id, symbol)

Оценка объема информации

PostgreSQL row overhead: 27 байт (заголовок heap tuple + выравнивание).

Получим следующие формулы для размера одной строки:

- users: $4 + 12 + 64 + 24 + 5 + 8 + \text{PgRowOverhead} = 117 + 27 = 144$
- user_watchlist: $4 + 4 + \text{PgRowOverhead} = 8 + 27 = 35$
- user_favorites: $4 + 4 + \text{PgRowOverhead} = 8 + 27 = 35$

- coins: $4 + 4 + 8 + 4 + 8 + \text{PgRowOverhead} = 28 + 27 = 55$
- coin_snapshots: $4 + 4 + 8 + 8 + 8 + 8 + 8 + \text{PgRowOverhead} = 48 + 27 = 75$
- statistics_settings: $4 + 4 + 16 + 8 + 8 + 8 + 8 + 8 + 5 + 8 + 8 + 8 + \text{PgRowOverhead} = 93 + 27 = 120$
- statistics_symbols: $4 + 4 + \text{PgRowOverhead} = 8 + 27 = 35$

Оценим общий размер (U -- пользователи, M -- монеты, S -- снимки на монету, R -- наборы статистики):

$$\text{PgBaseOverhead} = 8 * 1024 * 1024 \text{ байт} = 8 \text{ Мбайт}$$

$$\begin{aligned} V = & \text{PgBaseOverhead} \\ & + U * 144 \\ & + U * \text{AvgWatchlistSize} * 35 \\ & + U * \text{AvgFavoritesSize} * 35 \\ & + M * 55 \\ & + M * S * 75 \\ & + R * 120 \\ & + R * \text{AvgSymbolsPerStats} * 35 \end{aligned}$$

Для сравнения с MongoDB примем те же константы:

$$U = X$$

$$M = 50$$

$$S = 672$$

$$R = 3 * X$$

$$\text{AvgWatchlistSize} = 10$$

$$\text{AvgFavoritesSize} = 5$$

$$\text{AvgSymbolsPerStats} = 3$$

Тогда:

$$\begin{aligned}
V(X) &= 8.00 \text{ Мбайт} \\
&+ 50 * 55 \\
&+ 50 * 672 * 75 \\
&+ X * (144 + 10 * 35 + 5 * 35 + 3 * (120 + 3 * 35)) \\
\\
&= 8.00 \text{ Мбайт} + 2\,522\,750 \text{ байт} + X * 1\,344 \text{ байт} \\
&\sim 10.41 \text{ Мбайт} + X * 1.31 \text{ Кбайт}
\end{aligned}$$

Избыточность данных

Избыточность модели складывается из:

- Базовых накладных расходов PostgreSQL: **8 Мбайт**
- Накладных расходов PostgreSQL на строку: 27 байт на каждую строку
- Дополнительных таблиц-
связей user_watchlist, user_favorites, statistics_symbols
- Необходимости хранить по одной строке на каждый символ в персональных списках и наборах статистики

Минимальный (сырой) размер данных без технических накладных расходов:

$$\begin{aligned}
\text{RawSize}(X) &= 50 * 24 && \text{-- coins: symbol(4) + name(8) +} \\
&\text{cmc_id(4) + updated_at(8)} \\
&\quad + 50 * 672 * 44 && \text{-- snapshots: coin_id(4) +} \\
&\text{ts(8) + price(8) + mktCap(8) + vol(8) + pct(8)} \\
&\quad + X * 113 && \text{-- users: login(12) + hash(64)} \\
&\text{+ salt(24) + role(5) + created_at(8)} \\
&\quad + X * 10 * 8 && \text{-- watchlist: user_id(4) +} \\
&\text{symbol(4)} \\
&\quad + X * 5 * 8 && \text{-- favorites: user_id(4) +} \\
&\text{symbol(4)}
\end{aligned}$$

```

+ 3 * X * 89                                -- statistics_settings:
user_id(4) + name(16) + 2*Date(16) + 3*double(24) +
aggregation(5) + 3*Date(24)
+ 3 * X * 3 * 8                                -- statistics_symbols:
settings_id(4) + symbol(4)

```

$$= 1\,479\,600 + 572 * X$$

Отношение полезной нагрузки к полному объёму:

$$\text{RawSize}(X) / V(X) =$$

$$= (1\,479\,600 + 572 * X) / (10\,911\,358 + 1\,344 * X)$$

При больших X:

$$572 / 1\,344 \sim 0.43$$

То есть слой пользовательских данных в SQL-модели занимает больше служебного места из-за отдельных таблиц-связей и накладных расходов на строки.

Направление роста модели

dV/dU = 1 344 байт -- один пользователь с 3 наборами статистики

dV/dM = 50 455 байт -- одна новая монета с историей за 7 дней

dV/d(M*S) = 75 байт -- один снимок одной монеты

dV/dR = 225 байт -- один сохраненный набор статистики вместе со строками `symbols`

Рост также линейный. Основной фактор роста - coin_snapshots, но слой пользовательских данных при SQL-модели растёт быстрее, чем в MongoDB,

потому что один и тот же пользовательский контекст разложен по нескольким таблицам.

Примеры данных

Модель

- Пользователь user
- Администратор admin
- Монеты BTC и ETH
- Два снимка BTC и один снимок ETH
- Один сохраненный набор статистики для user

Представление модели в БД

```
INSERT INTO users (id, login, password_hash, password_salt,
role, created_at)
```

```
VALUES
```

```
                (1,                                'user',
'9e7f4cf8a1c29d4a8b0a558d22d37f1c6119d6ef4a72ec3b3c28f0b20ec77
a91',      'm4G7c2k9Qp1s8Zx5Vb3Nw0Yt',      'user',      '2026-03-27
09:00:00'),
```

```
                (2,                                'admin',
'0a1fb5bbd5c2a447da4f2752ae2f0d7fd8c964bc239be1bc7b1cf3477512e
81a',      'Qw8Yp2Ls4Nh7Jk1Vc5Rm9Tz0',      'admin',      '2026-03-27
09:05:00');
```

```
INSERT INTO coins (id, symbol, name, cmc_id, updated_at)
```

```
VALUES
```

```
  (1, 'BTC', 'Bitcoin', 1, '2026-03-27 12:00:00'),
  (2, 'ETH', 'Ethereum', 1027, '2026-03-27 12:00:00');
```

```

INSERT INTO user_watchlist (user_id, symbol)
VALUES
    (1, 'BTC'),
    (1, 'ETH'),
    (2, 'BTC'),
    (2, 'ETH');

```

```

INSERT INTO user_favorites (user_id, symbol)
VALUES
    (1, 'BTC'),
    (2, 'ETH');

```

```

INSERT INTO coin_snapshots (id, coin_id, timestamp, price,
market_cap, volume_24h, percent_change_24h)
VALUES
    (1, 1, '2026-03-27 11:45:00', 83400.25, 1651234567890,
43210987654, 1.35),
    (2, 1, '2026-03-27 12:00:00', 83512.80, 1653456789012,
44321098765, 1.49),
    (3, 2, '2026-03-27 12:00:00', 1945.60, 234567890123,
18765432109, -0.72);

```

```

INSERT INTO statistics_settings (
    id, user_id, name, time_from, time_to,
    min_price, max_price, min_volume, aggregation,
    created_at, updated_at, last_used_at
)
VALUES (
    1, 1, 'BTC vs ETH / week',

```

```
'2026-03-24 00:00:00', '2026-03-29 00:00:00',  
1000.0, 200000.0, 1000000.0, 'days',  
'2026-03-29 10:30:00', '2026-03-29 10:30:00', '2026-03-29  
11:55:00'  
);
```

```
INSERT INTO statistics_symbols (settings_id, symbol)  
VALUES  
  (1, 'BTC'),  
  (1, 'ETH');
```

Примеры запросов

UC-12. Регистрация

-- Шаг 1: проверить уникальность логина

```
SELECT id FROM users WHERE login = 'user';
```

-- Шаг 2: создать пользователя

```
INSERT INTO users (login, password_hash, password_salt, role,  
created_at)  
VALUES (  
  'user',  
  '9e7f4cf8a1c29d4a8b0a558d22d37f1c6119d6ef4a72ec3b3c28f0b20ec  
77a91',  
  'm4G7c2k9Qp1s8Zx5Vb3Nw0Yt',  
  'user',  
  NOW()  
);
```

Количество запросов: **2**, задействованных таблиц: **1** (users)

UC-13. Логин

-- Шаг 1: получить пользователя

```
SELECT id, login, password_hash, password_salt, role
FROM users
WHERE login = 'user';
```

-- Шаг 2: загрузить наборы статистики пользователя

```
SELECT *
FROM statistics_settings
WHERE user_id = 1
ORDER BY last_used_at DESC, updated_at DESC;
```

Количество запросов: 2, задействованных
таблиц: 2 (users, statistics_settings)

UC-1. Просмотр списка монет

```
SELECT c.symbol, c.name, cs.price, cs.percent_change_24h,
cs.market_cap, cs.volume_24h
FROM user_watchlist uw
JOIN coins c ON c.symbol = uw.symbol
JOIN coin_snapshots cs ON cs.coin_id = c.id
WHERE uw.user_id = 1
      AND cs.timestamp = (
        SELECT MAX(timestamp)
        FROM coin_snapshots
        WHERE coin_id = c.id
      )
ORDER BY cs.market_cap DESC;
```

Количество запросов: 1, задействованных
таблиц: 3 (user_watchlist, coins, coin_snapshots)

UC-2. Добавление монеты в список отслеживаемых

-- Шаг 1: проверить существование монеты

```
SELECT symbol FROM coins WHERE symbol = 'BTC';
```

-- Шаг 2: проверить, что такой монеты еще нет в списке
пользователя

```
SELECT 1
```

```
FROM user_watchlist
```

```
WHERE user_id = 1 AND symbol = 'BTC';
```

-- Шаг 3: добавить тикер в watchlist пользователя

```
INSERT INTO user_watchlist (user_id, symbol)
```

```
VALUES (1, 'BTC');
```

Количество запросов: 3, задействованных
таблиц: 2 (coins, user_watchlist)

UC-7. Добавление монеты в избранное

-- Шаг 1: проверить, что такой монеты еще нет в избранном

```
SELECT 1
```

```
FROM user_favorites
```

```
WHERE user_id = 1 AND symbol = 'BTC';
```

-- Шаг 2: добавить монету в избранное

```
INSERT INTO user_favorites (user_id, symbol)
```

```
VALUES (1, 'BTC');
```

Количество запросов: 2, задействованных
таблиц: 1 (user_favorites)

UC-8. Удаление монеты из избранного

```
DELETE FROM user_favorites  
WHERE user_id = 1 AND symbol = 'BTC';
```

Количество запросов: 1, задействованных
таблиц: 1 (user_favorites)

UC-10. Импорт и экспорт данных

Экспорт всех данных

```
SELECT * FROM users;  
SELECT * FROM user_watchlist;  
SELECT * FROM user_favorites;  
SELECT * FROM coins;  
SELECT * FROM coin_snapshots;  
SELECT ss.*, ARRAY_AGG(sym.symbol) AS symbols  
FROM statistics_settings ss  
LEFT JOIN statistics_symbols sym ON sym.settings_id = ss.id  
GROUP BY ss.id;
```

Количество запросов: 6, задействованных таблиц: 7

Импорт всех данных

```
TRUNCATE  
    statistics_symbols,  
    statistics_settings,  
    user_favorites,  
    user_watchlist,
```

```
coin_snapshots,  
coins,  
users  
CASCADE;
```

```
INSERT INTO users (...) VALUES ...;  
INSERT INTO user_watchlist (...) VALUES ...;  
INSERT INTO user_favorites (...) VALUES ...;  
INSERT INTO coins (...) VALUES ...;  
INSERT INTO coin_snapshots (...) VALUES ...;  
INSERT INTO statistics_settings (...) VALUES ...;  
INSERT INTO statistics_symbols (...) VALUES ...;
```

Количество запросов: 8, задействованных таблиц: 7

UC-11. Настройка и сохранение кастомной статистики

-- Шаг 1: построить агрегированный график

```
SELECT  
    c.symbol,  
    DATE_TRUNC('day', cs.timestamp) AS period,  
    AVG(cs.price) AS avg_price,  
    AVG(cs.volume_24h) AS avg_volume  
FROM coin_snapshots cs  
JOIN coins c ON c.id = cs.coin_id  
WHERE c.symbol IN ('BTC', 'ETH')  
    AND cs.timestamp BETWEEN '2026-03-24' AND '2026-03-29'  
    AND cs.price >= 1000  
    AND cs.price <= 200000  
    AND cs.volume_24h >= 1000000  
GROUP BY c.symbol, DATE_TRUNC('day', cs.timestamp)
```

```
ORDER BY period ASC, c.symbol ASC;
```

```
-- Шаг 2: сохранить набор настроек
```

```
INSERT INTO statistics_settings (  
    user_id, name, time_from, time_to,  
    min_price, max_price, min_volume, aggregation,  
    created_at, updated_at, last_used_at  
)  
VALUES (  
    1, 'BTC vs ETH / week',  
    '2026-03-24', '2026-03-29',  
    1000.0, 200000.0, 1000000.0, 'days',  
    NOW(), NOW(), NOW()  
)  
RETURNING id;
```

```
-- Шаг 3: сохранить список монет
```

```
INSERT INTO statistics_symbols (settings_id, symbol)  
VALUES (1, 'BTC'), (1, 'ETH');
```

Количество запросов: 3, задействованных
таблиц: 4 (coins, coin_snapshots, statistics_settings, statistics_symbols)

UC-11. Список валют из статистики

```
SELECT DISTINCT statistics_symbols.symbol  
FROM statistics_settings  
JOIN statistics_symbols  
    ON statistics_symbols.settings_id = statistics_settings.id  
WHERE statistics_settings.user_id = 1
```

```
ORDER BY statistics_symbols.symbol;
```

Количество запросов: 1, задействованных таблиц: 2 (statistics_settings, statistics_symbols)

UC-11. Максимальное изменение курса

```
WITH first_snapshots AS (  
    SELECT coin_id, MIN(timestamp) AS first_timestamp  
    FROM coin_snapshots  
    WHERE timestamp >= '2026-03-24' AND timestamp < '2026-03-29'  
    GROUP BY coin_id  
)  
last_snapshots AS (  
    SELECT coin_id, MAX(timestamp) AS last_timestamp  
    FROM coin_snapshots  
    WHERE timestamp >= '2026-03-24' AND timestamp < '2026-03-29'  
    GROUP BY coin_id  
)  
SELECT  
    coins.symbol,  
    first_prices.price AS first_price,  
    last_prices.price AS last_price,  
    ABS((last_prices.price - first_prices.price) /  
first_prices.price * 100) AS change_percent  
FROM coins  
JOIN first_snapshots  
    ON first_snapshots.coin_id = coins.id  
JOIN last_snapshots  
    ON last_snapshots.coin_id = coins.id  
JOIN coin_snapshots AS first_prices
```

```

ON first_prices.coin_id = first_snapshots.coin_id
AND first_prices.timestamp = first_snapshots.first_timestamp
JOIN coin_snapshots AS last_prices
ON last_prices.coin_id = last_snapshots.coin_id
AND last_prices.timestamp = last_snapshots.last_timestamp
ORDER BY change_percent DESC, coins.symbol ASC
LIMIT 10;

```

Количество запросов: 1, задействованных таблиц: 2 (coins, coin_snapshots)

3.3 Сравнение моделей

Объём информации

Модель	Базовый размер	На 1 пользователь	Формула роста
Нереляционная (MongoDB)	11.53 Мбайт	0.88 Кбайт	$11.53 \text{ Мбайт} + X * 0.88 \text{ Кбайт}$
Реляционная (PostgreSQL)	10.41 Мбайт	1.31 Кбайт	$10.41 \text{ Мбайт} + X * 1.31 \text{ Кбайт}$

SQL-модель компактнее для центральной части приложения - справочника монет и исторических снимков. После добавления пользовательских данных MongoDB оказывается выгоднее по приросту на одного пользователя: watchlist и favorites хранятся внутри документа пользователя, а не в отдельных таблицах-связях.

Эффективность хранения

Параметр	MongoDB	PostgreSQL
Технические накладные расходы базы	8 Мбайт	8 Мбайт
Накладные расходы на	50 байт	27 байт

Параметр	MongoDB	PostgreSQL
документ / строку		
Полезная нагрузка при больших X	~ 0.55	~ 0.43

Здесь эффективность хранения рассматривается по числу пользователей при больших значениях X. Для пользовательских данных MongoDB выгоднее, потому что данные одного пользователя хранятся в одном документе. В SQL та же информация распределена по нескольким таблицам, и каждая строка добавляет свои накладные расходы.

Масштабируемость (прирост на объект)

Сущность	MongoDB	PostgreSQL
Пользователь	904 В	1 344 В
Монета	72.28 Кбайт	49.28 Кбайт
Снимок цены	110 В	75 В
Набор статистики	195 В	225 В

Реляционная модель дешевле по монетам и снимкам. Нереляционная - дешевле по пользовательскому контексту и наборам статистики.

Запросы по сценариям использования

Юзкейс / действие	MongoDB (запросов / коллекций)	PostgreSQL (запросов / таблиц)
UC-1	2 / 2	1 / 3
UC-2	2 / 2	3 / 2
UC-7	1 / 1	2 / 1
UC-8	1 / 1	1 / 1
UC-10	8 / 4	8 / 7
UC-11	2 / 2	3 / 4
UC-12	2 / 1	2 / 1

Юзкейс / действие	MongoDB (запросов / коллекций)	PostgreSQL (запросов / таблиц)
UC-13	2 / 2	2 / 2
UC-11. Список валют из статистики	1 / 1	1 / 2
UC-11. Максимальное изменение курса	1 / 1	1 / 2

В MongoDB чтение watchlist и favorites опирается на один документ пользователя и одну выборку по историческим данным. В SQL такие сценарии требуют либо дополнительных таблиц-связей, либо операций соединения.

3.4 Вывод

Для CryptoWatch основным вариантом выглядит нереляционная модель.

SQL-модель лучше по объёму хранения временных рядов: одна монета с историей за 7 дней занимает меньше места, и один снимок цены тоже весит меньше. Но для пользовательских сценариев документная модель удобнее:

- учетная запись, watchlist и favorites хранятся в одном документе users
- пользовательские данные читаются без операций соединения
- импорт и экспорт можно выполнять напрямую по коллекциям

Поэтому выбор MongoDB выглядит обоснованным: она менее выгодна по памяти для централизованной истории цен, но лучше подходит для текущей архитектуры и персонализированных сценариев приложения.

4. РАЗРАБОТАННОЕ ПРИЛОЖЕНИЕ

4.1. Краткое описание

Разработанное приложение представляет собой веб-систему для отслеживания портфеля криптовалют, анализа котировок и управления пользовательскими данными, развертываемую с помощью Docker Compose и состоящую из трёх основных контейнеров: базы данных, серверной (backend) и клиентской (frontend) частей.

Контейнер базы данных: Обеспечивает хранение всей информации о пользователях, их аккаунтах, watchlist, избранном, метаданных криптовалют, исторических снимках котировок и пользовательских наборах статистики. Используется нереляционная СУБД MongoDB. При запуске контейнера происходит инициализация необходимых коллекций и индексов, что позволяет системе сразу приступить к работе.

Backend: Реализован на Java с использованием фреймворка Spring Boot. Backend запускается в контейнере на базе соответствующего образа с Java Runtime Environment. При инициализации создаются необходимые коллекции MongoDB, индексы и миграции схемы. Все необходимые зависимости управляются через систему сборки Gradle и включаются в артефакт приложения, который копируется в рабочую директорию контейнера. Запуск осуществляется стандартными средствами Spring Boot. Backend предоставляет REST API для взаимодействия с системой, включая:

- Аутентификацию и авторизацию пользователей с использованием JWT.
- Управление персональным портфелем: добавление/удаление монет в watchlist и избранное.
- Поиск, фильтрацию и сортировку криптовалют по цене, рыночной капитализации, объему торгов и изменению цены.
- Построение пользовательской и аналитики с различными периодами агрегации (часы, дни, недели).

- Сравнение монет с построением графиков и расчетом статистических показателей.
- Синхронизацию с API CoinMarketCap для получения актуальных котировок.
- Административные функции импорта/экспорта данных и управления системой.

Frontend: Это многостраничное веб-приложение, разработанное с использованием TypeScript, фреймворка Next.js и экосистемы сопутствующих библиотек:

- Next.js для server-side rendering и маршрутизации.
- React для построения пользовательского интерфейса.
- Tailwind CSS для стилизации.
- Shadcn/ui для готовых UI-компонентов.
- React hooks для управления состоянием компонентов.
- Встроенные типы TypeScript для типобезопасности.

Frontend разворачивается в Docker-контейнере с Node.js окружением. Зависимости из package.json устанавливаются, выполняется production-сборка, и статические ассеты раздаются через встроенный сервер Next.js. Клиентская часть взаимодействует с backend через REST API, отображая информацию о монетах, портфеле пользователя, исторической динамике цен и результатах аналитики в удобном и наглядном виде. Пользователь может регистрироваться, создавать и управлять watchlist и избранным, искать монеты, просматривать их детали и графики, проводить сравнение активов. Администраторы имеют доступ к функциям импорта/экспорта и контроля системы.

Вся система полностью контейнеризирована, что обеспечивает простоту развертывания, масштабируемость и независимость от среды пользователя. Это также упрощает обновление отдельных компонентов системы и интеграцию новых источников данных.

Ключевые функции:

- Фильтрация, поиск и сортировка монет по различным критериям.
- Таблицы с пагинацией для управления grandes наборами данных.
- Интерактивные графики котировок и статистики с помощью встроенных инструментов визуализации.
- Персональные пресеты аналитики для быстрого доступа к часто используемым расчетам.
- Синхронизация с внешним API для поддержки актуальных данных или работа на локальных снимках.

4.2. Используемые технологии

- СУБД: MongoDB
- Backend: Java + Spring Boot + Gradle
- Frontend: Next.js + React + TypeScript + Tailwind CSS + Shadcn/ui
- DevOps: Docker + Docker Compose

4.3 Ссылка на приложение

Ссылка на github-репозиторий: <https://github.com/moevm/nsql1h26-crypto>

4.4. Скриншоты приложения

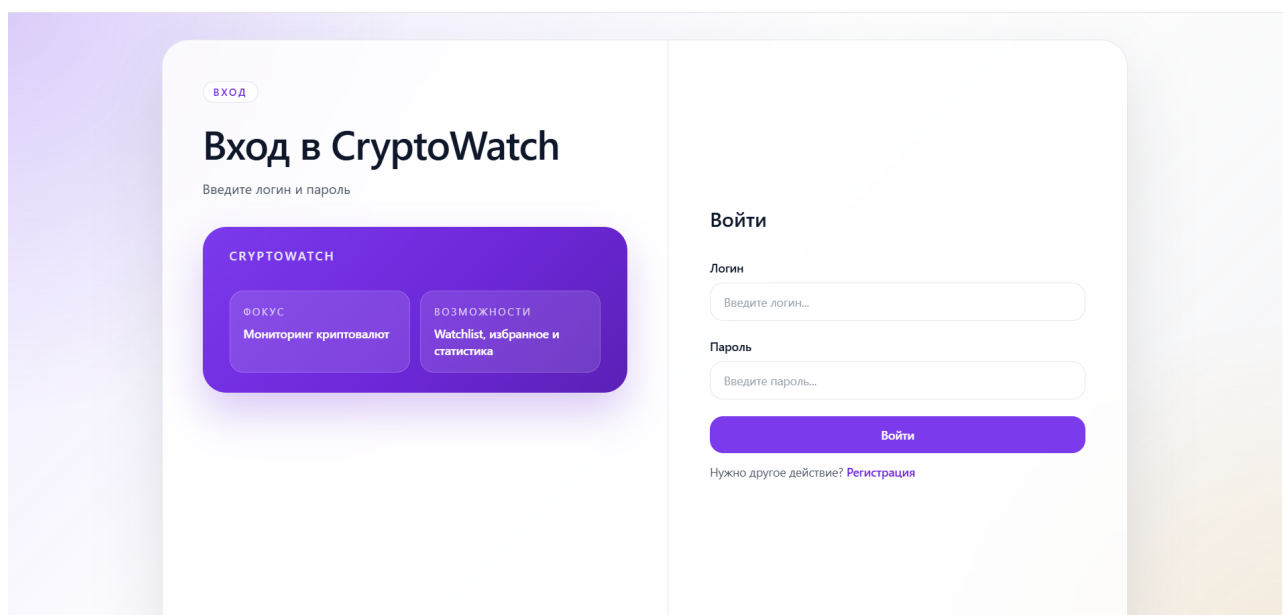


Рисунок 4 – страница входа

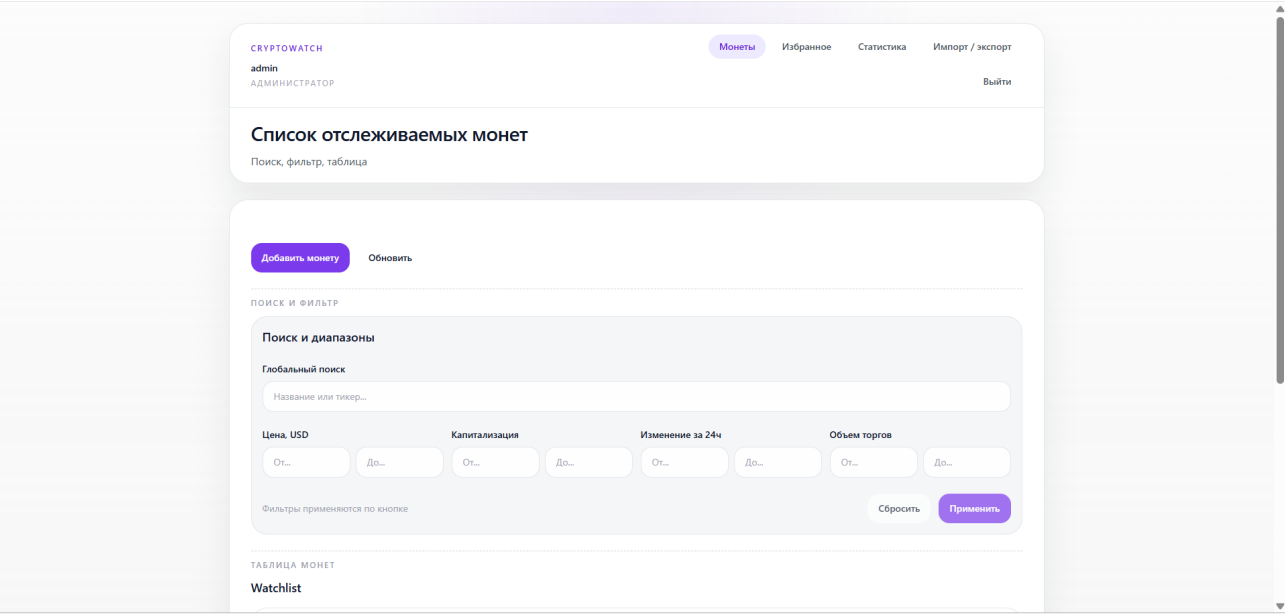


Рисунок 5 – главная страница

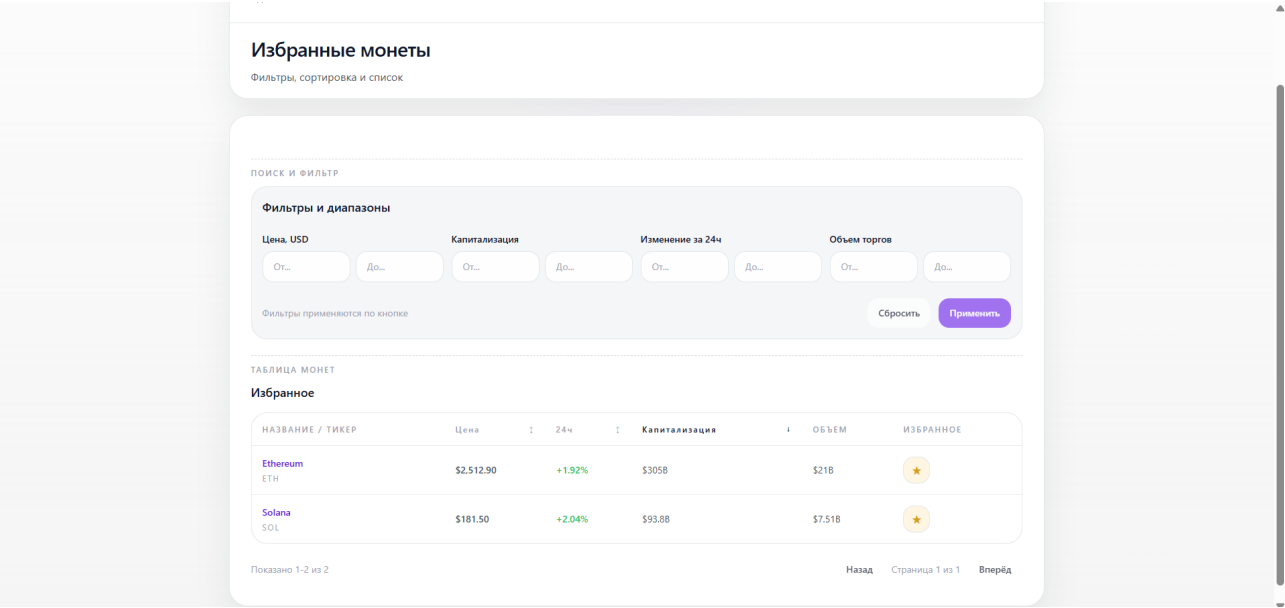


Рисунок 6 – страница избранного

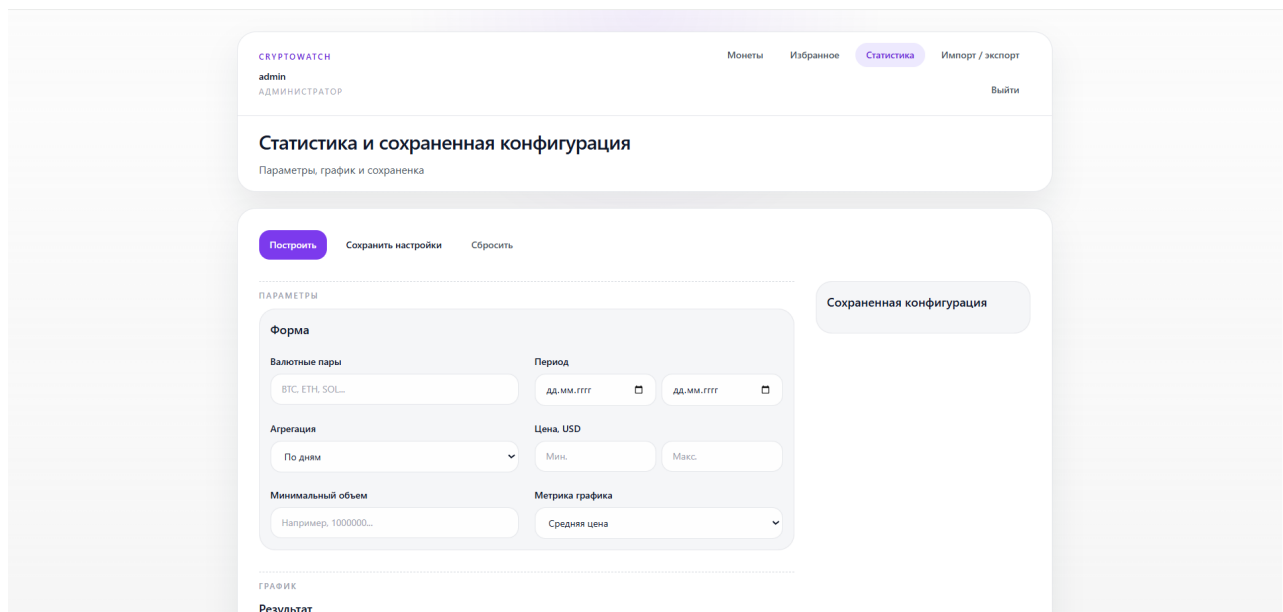


Рисунок 7 – страница статистики и сравнения

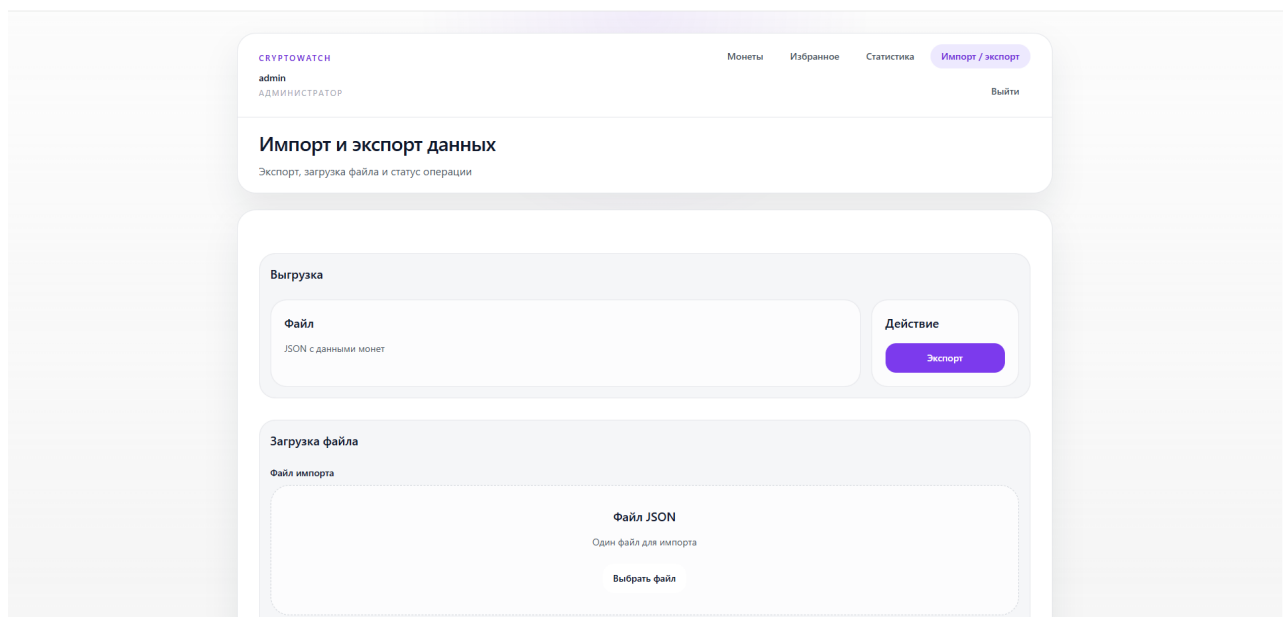


Рисунок 8 – страница импорта и экспорта

ЗАКЛЮЧЕНИЕ

5.1. Достигнутые результаты

В результате выполнения работы был разработан и реализован веб-сервис для мониторинга и анализа данных по криптовалютам, основанный на нереляционной модели хранения данных в MongoDB. В процессе работы была сформирована и применена структура базы данных для хранения информации о пользователях, ролях доступа, метаданных монет, исторических снимках котировок, watchlist, избранном и пользовательских статистических настройках. Для удобства развертывания, тестирования и эксплуатации системы использованы Docker и Docker Compose.

Фронтенд-часть приложения, построенная на Next.js, React, TypeScript и Tailwind CSS, обеспечивает многофункциональный и интуитивно понятный пользовательский интерфейс:

- Пользователи могут регистрироваться, авторизовываться и работать с персональными данными.
- Реализованы сценарии поиска, фильтрации, сортировки и просмотра списков криптовалют.
- Доступен просмотр детальной информации по монетам, включая исторические данные и графические представления.
- Реализованы пользовательские сценарии управления watchlist/избранным и работы со статистикой.
- Администраторам доступны функции управления данными, включая импорт/экспорт и эксплуатационные операции.

Бэкенд, реализованный на Java с использованием Spring Boot, инкапсулирует основную бизнес-логику и интеграционные возможности:

- Обработывает запросы клиентской части, выполняет валидацию данных и обеспечивает взаимодействие с MongoDB.
- Реализует аутентификацию и авторизацию с разграничением ролей.

- Предоставляет REST API для операций с монетами, портфелем, статистикой и административными функциями.
- Поддерживает интеграцию с внешним источником данных CoinMarketCap для обновления рыночной информации.
- Реализует серверные механизмы фильтрации, сортировки и пагинации для работы с большими наборами данных.

Взаимодействие между фронтендом и бэкендом осуществляется через REST API. Приложение поддерживает ключевые пользовательские и административные сценарии и обеспечивает основу для дальнейшего функционального развития.

5.2. Недостатки и пути для улучшения полученного решения

В процессе разработки и эксплуатации системы были выявлены направления, развитие которых позволит повысить устойчивость, удобство использования и долгосрочную эффективность решения:

- Потенциал расширения функциональной глубины: отдельные пользовательские сценарии могут быть дополнительно углублены для повышения аналитической ценности системы и поддержки более широкого круга практических задач.
- Необходимость дальнейшей унификации поведения компонентов: при росте функциональности важно поддерживать единообразие бизнес-логики, интерфейсных паттернов и правил обработки данных во всех модулях системы.
- Резерв для усиления отказоустойчивости критичных операций: процессы, связанные с массовой обработкой данных, требуют постоянного совершенствования механизмов проверки, контроля целостности и безопасного восстановления.

- Возможности улучшения пользовательского опыта: для повышения прозрачности работы системы целесообразно развивать качество обратной связи, навигационные сценарии и общую предсказуемость интерфейса.
- Перспективы оптимизации производительности при масштабировании: по мере роста объема данных и нагрузки актуальны регулярная оптимизация структуры хранения, индексации и наиболее ресурсоемких операций.

5.3. Будущее развитие решения

Для дальнейшего повышения ценности и зрелости сервиса возможны следующие направления развития:

- Расширение аналитического функционала: развитие сценариев сравнения монет, добавление новых типов графиков и углубленной статистики.
- Усиление надежности операций с данными: внедрение более строгих и безопасных flow импорта/экспорта, включая предварительную полную валидацию и механизмы отката.
- Повышение качества пользовательского опыта (UI/UX): дальнейшая доработка адаптивности, клавиатурной навигации, валидации форм и визуальной интерпретации данных.
- Оптимизация производительности и масштабирования: улучшение структуры индексов, оптимизация API-запросов и подготовка системы к росту числа пользователей и объема котировок.
- Развитие интеграций и эксплуатационных возможностей: расширение работы с внешними источниками рыночных данных, улучшение мониторинга и инструментов администрирования.

ПРИЛОЖЕНИЯ

6.1. Документация по сборке и разворачиванию приложения

Для сборки и разворачивания приложения используется контейнеризация через Docker и оркестрация с помощью docker-compose, что упрощает запуск и согласование сервисов. В корне репозитория находится файл docker-compose.yml, в котором описаны сервисы: база данных (MongoDB), backend (Java/Spring Boot) и frontend (Next.js).

Базовый сценарий разворачивания:

1. Установите Docker (Docker Desktop для Windows/macOS или Docker Engine для Linux) и убедитесь, что docker и docker-compose доступны в терминале.

Клонируйте репозиторий и перейдите в его корень:

```
git clone https://github.com/moevm/nsql1h26-crypto.git
cd nsql1h26-crypto
```

2. Соберите и запустите все сервисы:

```
docker compose up -d
```

3. После успешного запуска сервисы будут подняты в контейнерах; адрес и порт фронтенда указаны в docker-compose.yml (по умолчанию - http://localhost:3000).

Замечания по конфигурации и отладке:

- Просмотр логов всех сервисов:
`docker compose logs -f`
- Просмотр логов конкретного сервиса:
`docker compose logs -f <service-name>`
- Проверить статус контейнеров:
`docker compose ps`

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Документация MongoDB: <https://www.mongodb.com/docs/> - Основной ресурс по работе с СУБД MongoDB.
2. Документация Spring Boot: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/> - Официальное руководство по фреймворку Spring Boot.
3. TypeScript - Справочник: <https://www.typescriptlang.org/docs/handbook/intro.html> - Официальный справочник и руководства по языку TypeScript.
4. Документация Docker: <https://docs.docker.com/> - Официальная документация по Docker и Docker Compose.